Practical: Min, Max, Sum and Average using Parallel Reduction

Laboratory Practice V – Computer Engineering

1 Aim

Implement Min, Max, Sum and Average operations using Parallel Reduction.

1.1 Objective

To understand the concept of parallel reduction and how it can be used to perform basic mathematical operations on given data sets.

2 Theory

2.1 What is Parallel Reduction?

Parallel Reduction is a technique to **combine many values into a single result** using multiple threads working simultaneously.

Simple Example: Given an array [5, 2, 9, 1, 7, 6, 8, 3, 4], find the minimum value.

2.1.1 Sequential Approach (1 thread)

```
One thread checks all elements one by one:
5 -> 2 -> 9 -> 1 -> 7 -> 6 -> 8 -> 3 -> 4
                |
            min = 1    (9 comparisons)
```

2.1.2 Parallel Reduction Approach (multiple threads)

```
Thread 1: [5, 2, 9]  -> local min = 2
Thread 2: [1, 7, 6]  -> local min = 1
Thread 3: [8, 3, 4]  -> local min = 3
           |
    REDUCE: min(2, 1, 3) = 1    (faster!)
```

Each thread finds a **local result**, then OpenMP **combines (reduces)** all local results into one **final result**.

2.2 How Reduction Works in OpenMP

The `reduction` clause in OpenMP:

1. Creates a **private copy** of the reduction variable for each thread.
2. Each thread works on its own private copy independently.
3. After all threads finish, OpenMP **combines** all private copies using the specified operator.

Syntax:

```
#pragma omp parallel for reduction(operator : variable)
```

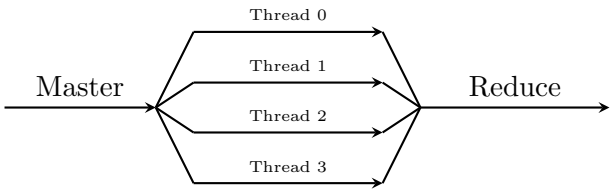
2.2.1 Supported Reduction Operators

Operator	Operation	Initial Value
`min`	Minimum	`INT_MAX` (or first element)
`max`	Maximum	`INT_MIN` (or first element)
`+`	Sum	0
`*`	Product	1

2.3 Concept of OpenMP

OpenMP (Open Multi-Processing) is an API for shared-memory parallel programming in C/C++/Fortran. It uses `#pragma` compiler directives and follows a **Fork-Join** execution model.

2.3.1 Fork-Join Model



FORK **JOIN + REDUCE**

2.4 How Each Operation Works

2.4.1 Min Reduction

```
Array: [1, 2, 3, 4, 5]

Thread 1: checks [1, 2]  -> local min = 1
Thread 2: checks [3, 4]  -> local min = 3
Thread 3: checks [5]     -> local min = 5

OpenMP REDUCE: min(1, 3, 5) = 1
Output: "The minimum value is: 1"
```

2.4.2 Max Reduction

```
Array: [1, 2, 3, 4, 5]

Thread 1: checks [1, 2]  -> local max = 2
Thread 2: checks [3, 4]  -> local max = 4
Thread 3: checks [5]     -> local max = 5

OpenMP REDUCE: max(2, 4, 5) = 5
Output: "The maximum value is: 5"
```

2.4.3 Sum Reduction

```
Array: [1, 2, 3, 4, 5]

Thread 1: sum of [1, 2]  -> local sum = 3
Thread 2: sum of [3, 4]  -> local sum = 7
Thread 3: sum of [5]     -> local sum = 5

OpenMP REDUCE: 3 + 7 + 5 = 15
Output: "The summation is: 15"
```

2.4.4 Average Reduction

```
Step 1: Find sum using reduction(+) = 15
Step 2: Divide by count: 15 / 5 = 3.0

Output: "The average is: 3"
```

3 Code

Listing 1: Min Max Sum Average using Parallel Reduction

```cpp
#include<iostream>
#include<omp.h>

using namespace std;

// Find MINIMUM using parallel reduction
int minval(int arr[], int n){
    int minval = arr[0];
    #pragma omp parallel for reduction(min : minval)
    for(int i = 0; i < n; i++){
        if(arr[i] < minval) minval = arr[i];
    }
    return minval;
}

// Find MAXIMUM using parallel reduction
int maxval(int arr[], int n){
```

```
    int maxval = arr[0];
    #pragma omp parallel for reduction(max : maxval)
    for(int i = 0; i < n; i++){
        if(arr[i] > maxval) maxval = arr[i];
    }
    return maxval;
}

// Find SUM using parallel reduction
int sum(int arr[], int n){
    int sum = 0;
    #pragma omp parallel for reduction(+ : sum)
    for(int i = 0; i < n; i++){
        sum += arr[i];
    }
    return sum;
}

// Find AVERAGE using sum reduction
double average(int arr[], int n){
    return (double)sum(arr, n) / n;
}

int main(){
    int n = 5;
    int arr[] = {1,2,3,4,5};
    cout << "The minimum value is: " << minval(arr, n) << '\n';
    cout << "The maximum value is: " << maxval(arr, n) << '\n';
    cout << "The summation is: " << sum(arr, n) << '\n';
    cout << "The average is: " << average(arr, n) << '\n';
    return 0;
}
```

4 Code Explanation

4.1 minval() Function

1. Initialize `minval = arr[0]` (first element).
2. `#pragma omp parallel for reduction(min : minval)` — divides array among threads, each finds local minimum.
3. OpenMP combines all local minimums using `min` operator.
4. Returns the global minimum.

4.2 maxval() Function

Same as `minval()`, but uses `reduction(max : maxval)` to find the **maximum** value.

4.3 sum() Function

1. Initialize `sum = 0`.

2. `#pragma omp parallel for reduction(+ : sum)` — each thread calculates partial sum.
3. OpenMP adds all partial sums using `+` operator.
4. Returns the total sum.

4.4 average() Function

Calls `sum()` to get the total sum, then divides by `n` (number of elements).

$$\text{Average} = \frac{\text{Sum}}{n} = \frac{15}{5} = 3.0$$

4.5 main() Function

Initializes array {`1, 2, 3, 4, 5`} and calls all four functions to print Min, Max, Sum, and Average.

5 Output

```
The minimum value is: 1
The maximum value is: 5
The summation is: 15
The average is: 3
```

6 Advantages and Limitations

6.1 Advantages

- Significantly faster for **large arrays** (work divided among threads).
- Simple to implement — just add `reduction` clause.
- **Thread-safe** — OpenMP handles synchronization automatically.
- Scales with number of CPU cores.

6.2 Limitations

- Thread creation overhead may reduce benefit for **small arrays**.
- Limited to **associative and commutative** operations (min, max, +, *).
- Performance depends on number of available cores.

7 Compilation

Listing 2: Compile and Run

```
g++ -fopenmp ParallelReduction.cpp -o reduce
./reduce
```

Note: The `-fopenmp` flag enables OpenMP reduction clause. Without it, `#pragma omp` directives are ignored and code runs sequentially (output remains correct but no parallelism).

8 Conclusion

In this practical, we implemented Min, Max, Sum, and Average operations using Parallel Reduction with OpenMP. The `#pragma omp parallel for reduction` clause was used to divide array processing among multiple threads, where each thread computes a local result and OpenMP automatically combines them into the final result. This technique is efficient for performing mathematical operations on large datasets using multi-core processors.